OfferPilot: the right offer, only when it pays

Hackathon PS-2 · Samiya Sehgal (2302662) · 26 Oct 2026 · Live demo page: https://claude.ai/artifact/UEaaWH4AHrrowPeRVQNbEu · Code: github.com/SamiyaS5904/Hackathon-PS_2

1. The problem

E-commerce platforms send offers to millions of customers, but not every offer is worth sending. A discount that wins back a customer who is about to leave is valuable; the same discount sent to someone who would have bought anyway is lost margin. **Task:** build a system that decides which offer each customer should get, and when, and delivers it to classmates through a Telegram bot. Generate realistic customer data. Rules we design set the limits; an AI agent may choose and personalise offers within them; **every AI choice must be checked against the rules before it is sent.** Also: work out what data a platform collects, how customers differ, which offer suits whom, what limits a business sets, and how to know whether offers work.

2. Our approach in one line: the AI chooses, code protects the money

Step	What happens	Who does it
1 Data	3,000 synthetic customers, 12 months: orders, browsing, carts, wallet, points, 19,663 past campaign offers with a random 'not sent' group. 8 hidden customer types.	Code (no AI)
2 Profile	Per customer: usual order gap, how late they are, favourite items, interest (cart / search / views), chance to buy anyway, full-price habit.	Code
3 Moment	Habit break, cart left, product interest, points expiring, idle wallet, or the manager clicks Run AI agent.	Code
4 Menu	6-10 personalised offers, each priced (expected profit) and rule-checked before the AI sees them, so the AI usually decides in one call.	Code
5 Decide	Pick / adapt an offer, send nothing, or wait and check again. Writes 2-3 message versions (Hinglish for Hindi speakers) and plain reasoning for the manager.	AI (Claude Sonnet 5.5)
6 Check	About 30 rules: profit, 10% margin floor, category caps, budgets, 2 offers per week, quiet hours, habit protection, honest message. Rejected twice → free fallback rules.	Code (final say)
7 Deliver	Telegram bot sends approved offers to linked testers; the dashboard shows every decision and why.	Bot + dashboard

3. Why this is better than the usual ways

Approach	Personal	Profit checked	Says why	Knows when not to send	Proven
Send everyone a coupon	No	No	No	No	No
Fixed rule tables	Little	Partly	Rule name only	Partly	Rarely
CRM tools (CleverTap, MoEngage)	Segments	No: marketer decides	No	Marketer's job	A/B tests
An AI agent alone	Yes	No: AI can do bad maths	Yes	Unreliable	No
OfferPilot	**Per customer**	**Every offer, in code**	**Full reasoning**	**Nothing / wait**	**193 tests + referee**

Profit is maths, never the AI: expected profit = chance to buy with the offer × margin left − chance to buy anyway × normal margin. A discount to someone who would buy anyway counts as a loss, so it's refused. Since v2 a lapsing customer's next 12 weeks count too (**win-back offers**, own budget), and full-price buyers get no coupons (**habit protection**).

4. How we researched and checked it

- **Learned from past campaigns:** random 'not sent' groups show who buys because of an offer. Our first guesses rated points 2x too high and VIP discounts too low; we recalibrated on the data.
- **Hidden-truth referee:** each synthetic customer has a hidden true behaviour the AI never sees. Every approach is scored against it, over 48 hours and over 12 weeks (habits break, coupons teach waiting, win-backs can revive a habit).
- **Attacks:** we tried injected instructions in a customer's name, 90% discounts, fake urgency and wrong amounts in messages. All were blocked by code.
- **Findings:** over 90 days, festival season made even loyal regulars look deal-driven, so habit protection uses 12 months. A ₹300 win-back cap made the most valuable VIP win-backs impossible, so it became ₹500.

5. What the marketing manager sees (dashboard at http://127.0.0.1:8010)

Page	What it is for
Today	Greeting, 'Plan your day' (how many AI decisions to allow, with the cost), key numbers, last 7 days.
Decisions	Every decision; click one to read why: facts used, options rejected with their numbers, normal or win-back, every rule check, the exact Telegram message and AI cost.
Customers	A customer's profile as the AI sees it; Run AI agent, Free rules only, simulate a cart / search.
Testers	Pick a profile to play, link a Telegram user with a QR code, re-link to another profile.
Guardrails / Rules	What each rule caught, budgets, circuit breaker, test proof / edit limits (each save is a new version; unsafe values are refused).

6. How a message reaches Telegram (and why there is no Send button)

- **Link once:** Dashboard → Testers → enter the tester's Telegram username, pick a customer profile to play → QR code / link → the tester presses Start in **@offer_pilot_samiya_bot**. A bot can't message anyone first.
- **Decide:** Dashboard → Customers → open the customer → **Run AI agent** (or Free rules only).
- **Sent automatically:** once the rules approve an offer, the bot program (python -m bot.telegram_bot, keep it running) picks it up within 15 seconds and sends it. The status becomes 'Sent on Telegram'. Offers decided between 22:00 and 08:00 wait until 08:00. Customers not linked to a tester show 'Would send'. There is **no manual Send button by design**: nothing reaches a customer without passing the rules.
- **Re-link demo:** switch the same Telegram to another profile (e.g. loyal regular → about-to-leave VIP), run again, and a different offer arrives with its own reasoning.

7. The AI, the trained model and the API cost

Part	What it is	Cost
Decision AI	Claude Sonnet 5.5 via the Anthropic API, used in one file only (engine/agent.py). Effort 'medium', 30 s timeout, usually one call per decision.	₹1.6-2.4 per decision
Daily cap	'Plan your day' on the dashboard sets how many AI decisions are allowed today (default 50, hard ceiling 50). Over the cap, or without a key, the free rules decide.	At most about ₹120 a day
Prediction model	Uplift model: two scikit-learn gradient-boosting models (chance to buy with / without an offer), trained on 19,663 past campaign offers (3,910 'not sent'). Accuracy for 'buys anyway' AUC 0.85. Averaged with an expert formula, which ranked customers best. Command: python -m engine.uplift --train (~2 min).	Free (no API)
Testing spend	All paid AI evaluation runs so far (profit simulations + 16-scenario checks). Saved answers make re-runs free.	About ₹200 in total

8. Where everything is on the laptop

What	Where
Synthetic data (SQLite, 168 MB)	D:\Hackathon PS_2\data\offerpilot.db (old version: data\offerpilot_v1_backup.db)
Data generator / CSV importer	data\generate.py / data\import_csv.py (sample brand: docs\csv_import\)
Trained model	models\uplift.joblib
Rules (editable on the Rules page)	config\rules.yaml → checked by engine\rules.py
Dashboard + API	python -m uvicorn api.main:app --port 8010 → http://127.0.0.1:8010
Evidence of real runs	docs\evidence\ · build log: documentation.md

9. Results (honest)

Check	Result
True profit vs sending everyone an offer, 48 hours (v1)	AI + rules **₹1,039** vs ₹827 (24 customers)
True profit over 12 weeks (v2, stricter referee)	AI + rules ₹1,515 vs ₹2,692: sending to everyone still wins here
Full-price share of orders in the next 12 weeks	AI + rules 59% vs send-to-all 55% (no offers: 65%)
AI behaviour in 16 golden scenarios	14/16 (pass bar 14), reasoning complete in all 16
Automatic tests / approved offers that lose value	193 pass / 0 with negative total value; 10% margin floor always holds

10. What is left, and how we will do it

- **Classmates on Telegram:** everything is ready (TESTING.md, profile picker, QR links). Needs 2-3 friends' usernames: link each to a different profile, run the same moment, then the re-link demo.
- **Beat 'send to everyone' over 12 weeks:** some VIPs whose habit has broken still look on schedule. Add a 'falling order frequency' signal so win-backs start earlier, then re-test.
- **Bigger samples and a real shop:** 24 customers per AI check keeps cost low but is noisy. The CSV importer (learning mode, 10% holdout) is ready for a real brand's export to measure real lift.
- **Measure real redemptions:** there is no store, so nobody actually redeems an offer. The message version sent is stored per decision; a store or coupon-code integration would show which offers and versions actually convert.